# LYNQ

Plataforma inteligente de empleabilidad: funcionalidades y decisiones técnicas

**Jorge Flores** · jfflores@udesa.edu.ar
Maestría en Inteligencia Artificial, Universidad de San Andrés
Proyecto del *New Business Launch* (NBL), en conjunto con el Executive MBA
Tatiana Ibañez · tibanez@udesa.edu.ar Melani Huetagoyena Rija · mhuetagoyena@udesa.edu.ar

Octubre de 2026 · versión 3.1.5

## 1. Qué es LYNQ

LYNQ nace como trabajo final del programa *New Business Launch* de la Universidad de San Andrés, desarrollado en conjunto por el Executive MBA, que aporta el caso de negocio (cliente, propuesta de valor, MVP y modelo de negocios), y la Maestría en Inteligencia Artificial, que aporta la plataforma. Su lema es *"Que buscar trabajo no sea un trabajo"*, y se define como la primera plataforma inteligente de empleabilidad de LATAM: un marketplace de dos vías que transforma la búsqueda laboral en un proceso predictivo, transparente y automatizado.

**El problema.** Del lado del candidato: portales fragmentados (el 100 % de los entrevistados usa más de tres), carga repetida de perfiles, CV adaptado a mano, búsquedas activas de 3 a 6 meses y ningún entrevistado conocía el rango salarial antes de recibir una oferta. Del lado de la empresa: datos salariales desactualizados o comprados a consultoras caras, sin *time-to-fill* predictivo, y un screening manual que consume entre el 30 % y el 50 % del tiempo de RRHH. En 18 entrevistas en profundidad (12 candidatos, 6 empresas), el 83 % de los candidatos y 5 de 6 responsables de RRHH validaron el problema; lo más valorado fue la transparencia salarial y el LYNQ Score del lado B2C, y la *shortlist* y el *time-to-fill* del lado B2B.

**Cliente objetivo.** Fase 1: profesionales de 22 a 40 años en perfiles no técnicos (administración, legales, comercial, marketing, finanzas) y startups/scale-ups de 150 a 400 empleados que contratan de 3 a 8 posiciones por mes y quieren bajar el *time-to-fill* de 6 semanas a menos de 3.

**Modelo de negocios.** Freemium B2C más SaaS B2B: LYNQ Free (USD 0), LYNQ Pro (USD 9,99/mes), Business Starter/Growth/Enterprise (USD 290, 790 y desde 1.900/mes) y LYNQ Data API (desde USD 500/mes). La transparencia salarial se mantiene gratis siempre. El break-even estimado son 50 cuentas B2B y 1.300 suscriptores Pro.

## 2. Funcionalidades

La plataforma implementa hoy las piezas centrales de la propuesta de valor, divididas por rol:

- **Candidato.** Registro, carga de CV (PDF/DOCX) que se convierte a JSON estructurado con un LLM, extracción automática de skills y de *similarity tags*; buscador unificado de avisos propios y scrapeados (Bumeran y Computrabajo); **LYNQ Score** de 0 a 100 por aviso; postulación con el CV elegido; **CV Tailor**, un agente conversacional que adapta el CV a un aviso; sugerencias de *upskilling* con cursos de Udemy; analíticas personales: posición entre los postulantes de un aviso, salario esperado contra el mercado y contra pares, *benchmark* entre perfiles parecidos, cobertura de skills y qué skills faltantes desbloquean más avisos.
- **Empresa.** Publicación y gestión de avisos; listado de candidatos ordenado por score con una explicación generada (fortalezas y preocupaciones); *time-to-fill* estimado a partir de avisos similares; panel con estado, postulaciones y score mediano de cada aviso.
- **Plataforma.** Mercado: avisos abiertos, top 10 de skills más demandadas con variación semanal, salarios por categoría y modalidad, avisos publicados por semana (12 semanas). Un *feeder* diario

mantiene el corpus de avisos externos vivo y verifica cuáles siguen publicados.

## 3. Arquitectura

LYNQ es un monorepo de ocho servicios desplegados en Kubernetes (EKS), más dos frontends servidos desde Cloudflare Workers (Figura 1). Las decisiones estructurales son tres: **un único punto de entrada** (el BFF es el único servicio con Ingress; el resto solo recibe tráfico interno del cluster), **un servicio por dominio** con su propio schema MySQL y **Python para todo lo que toca un LLM**, Java/Spring para el dominio transaccional.

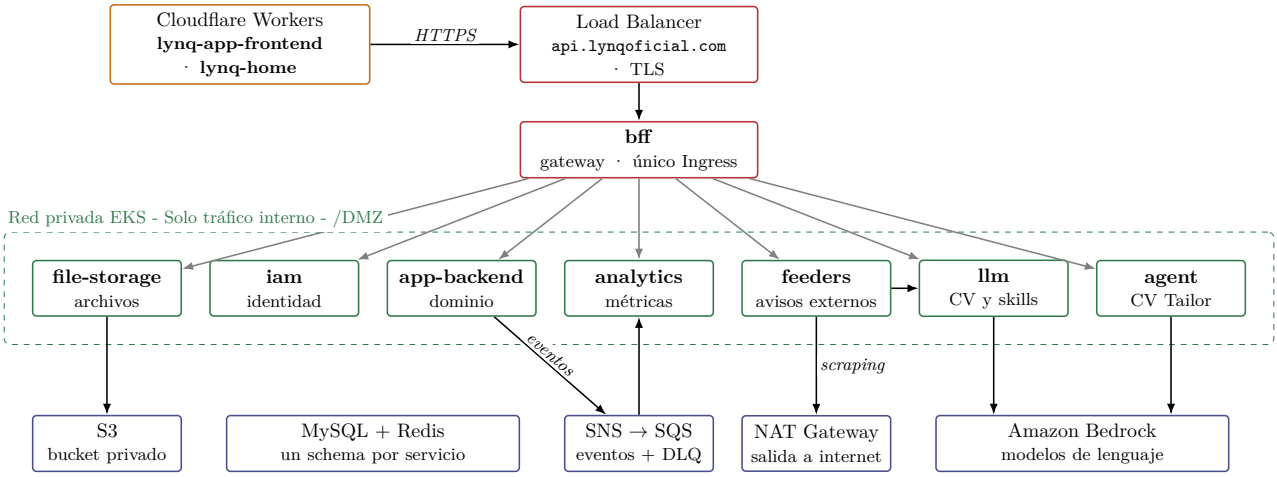



Figura 1: Topología de producción. Solo los Workers y el Load Balancer reciben tráfico de internet; el BFF es el único servicio expuesto y relaya a los siete microservicios, que se alcanzan únicamente por red interna de Kubernetes. Todos los servicios de dominio usan MySQL y Redis; solo `file-storage` toca S3, solo `llm` y `agent` llaman a Bedrock, y `feeders` es el único que sale a internet, por la NAT.

| Servicio | Stack | Puerto | Responsabilidad |
|---|---|---|---|
| lynq-bff | Spring Boot | 8080 | Único Ingress. Verifica la firma del JWT, relaya cada ruta a su servicio, orquesta flujos compuestos (importar CV, CV Tailor). |
| lynq-iam | Spring Boot | 8080 | Usuarios, roles, emisión y validación de JWT, refresh tokens en Redis. |
| lynq-app-backend | Spring Boot | 8080 | Avisos, empresas, CVs, postulaciones, LYNQ Score, publicación de eventos de dominio, ingesta de avisos externos. |
| lynq-file-storage | Spring Boot | 8080 | Metadata de archivos y URLs pre-firmadas de S3. Único servicio con credenciales de S3. |
| lynq-analytics | Spring Boot | 8080 | Consume eventos, proyecta un read model y sirve las analíticas; snapshot diario. |
| lynq-llm | FastAPI | 8084 | Parseo de CV, extracción de skills y tags, explicaciones, traducción, cursos. Bedrock Nova Lite. |
| lynq-agent | FastAPI + LangGraph | 8090 | CV Tailor: grafo multiagente sobre Bedrock Nova Pro, con persistencia y traza propia. |
| lynq-feeders | FastAPI | 8089 | Scraping de Bumeran y Computrabajo, enriquecimiento con lynq-llm, verificación de vigencia. |

Cuadro 1: Servicios del monorepo. Infraestructura como código con Terraform (VPC, EKS, S3, SNS/SQS, IAM) y Helm (deployments, CronJobs); LocalStack y docker-compose replican el stack en local.

# 4. Autenticación y autorización

**Tokens propios, sin proveedor externo.** La propuesta de valor prevé *login* social, y el frontend ya muestra el botón de Google, pero el flujo OAuth 2.0/OIDC con Google no está implementado todavía: no hay *authorization code*, *id_token* ni alta automática de usuarios. Lo que existe es un esquema de dos tokens emitido por `lynq-iam`:

1. **Registro** (`POST /auth/register`): `{username, password, email, role}`, con rol `R_CANDIDATE` o `R_COMPANY`. La contraseña se almacena con BCrypt (*strength* 10). El id del usuario es un UUIDv7 y es el mismo en todos los servicios.
2. **Login** (`/auth/login/email` o `/username`): devuelve un *access token* JWT y un *refresh token*. El JWT lleva `jti`, `sub` (userId), `username`, `email`, `roles`, `iat` y `exp`, se firma con HMAC sobre un secreto compartido (`JWT_SECRET`) y vence a los **15 minutos**. El refresh token es opaco: 64 bytes de `SecureRandom` en Base64 URL-safe, guardado en Redis como `refresh:{token} = userId` con TTL de **30 días**.
3. **Refresh** (`POST /auth/refresh` con `Authorization: Bearer {refreshToken}`): si el token existe en Redis se relee el usuario de MySQL (los roles se refrescan) y se emite un JWT nuevo. El refresh token no rota.
4. **Relay en el BFF.** Tres filtros en orden: `RequestUuidFilter` (exige `lynq-request-uuid`, el id de correlación que viaja a todos los servicios), `AuthHeaderExistenceFilter` (401 sin `Authorization`) y `JwtSignatureFilter`, que verifica la firma *localmente* con el mismo secreto y arma un `LynqUserPrincipal`. Las únicas rutas anónimas son registro, login y los *checks* de disponibilidad; `/auth/refresh` exige header pero no firma.
5. **Validación en cada servicio.** Los servicios de dominio no verifican la firma: llaman a `GET /auth/user-info` del IAM con el bearer que les reenvió el BFF (`IamAuthenticationFilter` en Java, `src/security` en Python) y construyen el principal con los roles que devuelve. Es una llamada más por request, a cambio de que el secreto viva solo en el IAM y el BFF.
6. **Autorización.** `@HasRole(CANDIDATE|COMPANY)` se declara en el servicio dueño del dato, bajo `/dmz/**`. Las rutas `/internal/**` (ingesta, replay, snapshot, score en lote) no pasan por el BFF: las protege un `InternalTokenFilter` con el header `lynq-internal-token`.

En el frontend, con *remember me* el refresh token va a `localStorage` y la sesión se rehidrata al recargar; sin él, todo vive en `sessionStorage`. Ante un 401, `authFetch` refresca y reintenta una vez.

# 5. LYNQ Score y similitud

## 5.1. LYNQ Score

Se calcula en `lynq-app-backend` (`LyNQScoreCalculator`) en cada request y nunca se persiste en el dominio, para que el cálculo exista en un solo lugar. Sea $V$ el vocabulario del candidato, la unión de sus skills y sus *similarity tags* normalizadas (*trim* y minúsculas); $S$ las skills que pide el aviso y $T$ sus tags. Se define la cobertura de un conjunto requerido $R$ como

$$\mathrm{cov}(R) = \mathrm{round}\left(100 \cdot \frac{|R \cap V|}{|R|}\right), \qquad \mathrm{cov}(\varnothing) = 0,$$

y el score como

$$\mathrm{score} = \begin{cases} 0 & \text{si } V = \varnothing \text{ o } S \cup T = \varnothing, \\ \max\left(\mathrm{cov}(S),\ \mathrm{cov}(T)\right) & \text{en otro caso.} \end{cases}$$

No hay pesos: gana la mejor de las dos coberturas. Las skills son términos concretos ("Spring Boot"); las tags son conceptos en inglés que lynq-llm genera para aviso y CV con el mismo prompt ("backend development"), así que $\mathrm{cov}(T)$ rescata candidatos cuyo CV usa otras palabras. El score viaja dentro del evento `ApplicationSubmitted` y también se expone en `POST /internal/score/batch` para el snapshot de analytics.

## 5.2. Similitud entre avisos y entre candidato y aviso

Las analíticas comparan contra "avisos similares" o "candidatos parecidos". La similitud vive en `lynq-analytics` sobre las tags y pondera cada tag por su rareza, con una variante de IDF recalculada en cada snapshot sobre los $N$ avisos del read model:

$$w(t) = \ln\left(\frac{N+1}{\mathrm{df}_t + 1}\right),$$

donde $\mathrm{df}_t$ (*document frequency*) es la cantidad de avisos que llevan la tag $t$. Es la *inverse document frequency* (IDF) clásica de recuperación de información: una tag vale más cuanto más rara es en el corpus. Con $N = 200$ avisos, una tag presente en 180 de ellos pesa casi nada, mientras que una que aparece en 5 pesa 35 veces más:

| Tag | Avisos que la llevan ($\mathrm{df}_t$) | Peso $w(t)$ |
|---|---|---|
| `teamwork` | 180 | $\ln(201/181) \approx 0{,}10$ |
| `backend development` | 40 | $\ln(201/41) \approx 1{,}59$ |
| `kubernetes` | 5 | $\ln(201/6) \approx 3{,}51$ |

Así, dos avisos que comparten solo `teamwork` no se consideran parecidos, porque casi todos la tienen; dos que comparten `kubernetes` sí. Sin esta ponderación, ambas coincidencias contarían igual y el conjunto de "avisos similares" estaría lleno de ruido. El +1 en numerador y denominador es un suavizado que evita el logaritmo de cero. Los pesos se recalculan en cada snapshot diario y se guardan en `tag_frequency`; una tag desconocida recibe el peso máximo observado. Sea $m$ la mediana de los pesos *por ocurrencia* (cada tag cuenta $\mathrm{df}_t$ veces), que representa "cuánto vale una tag típica".

**Aviso contra aviso** (time-to-fill, salario de la posición): Jaccard ponderado entre las tags $A$ del aviso de referencia y las $B$ del otro,

$$J(A,B) = \frac{\sum_{t \in A \cap B} w(t)}{\sum_{t \in A \cup B} w(t)}, \qquad \theta_k = \min\left(1,\ \frac{k \cdot m}{\sum_{t \in A} w(t)}\right),$$

y se admite el aviso si $J(A,B) \geq \theta_k$.

Un ejemplo con los pesos de la tabla anterior, más `java` (1,20), `sql` (0,90), `microservices` (2,30), `excel` (0,60) y `communication` (0,15). El aviso de referencia es $A = \{$`backend development`, `kubernetes`, `java`, `teamwork`$\}$, con $\sum_{t \in A} w(t) = 6{,}40$, y se lo compara con otros dos que comparten exactamente dos tags con él:

| Aviso | Tags | En común | Jaccard simple | $J$ ponderado |
|---|---|---|---|---|
| $B$ | `backend development`, `kubernetes`, `sql`, `microservices` | 2 | $2/6 = 0{,}33$ | $5{,}10/9{,}60 = 0{,}53$ |
| $C$ | `teamwork`, `java`, `sql`, `excel`, `communication` | 2 | $2/7 = 0{,}29$ | $1{,}30/8{,}05 = 0{,}16$ |

Contando tags, $B$ y $C$ se parecen a $A$ casi lo mismo. Ponderando, $B$ comparte las dos tags que más pesan y $C$ las dos que menos: con $m = 1{,}2$ el umbral es $\theta_2 = \min(1,\ 2 \cdot 1{,}2/6{,}40) = 0{,}375$, así que $B$ entra al conjunto de avisos similares y $C$ queda afuera.

**El umbral no es un número fijo.** $\theta_k$ se calcula para cada aviso de referencia y significa siempre lo mismo: compartir el equivalente a $k$ tags de peso típico, con $k = 2$ (`thresholdTags`). Como $J$ es una fracción sobre la unión, ese mismo requisito se traduce en un valor distinto según cuántas tags tenga $A$:

| Aviso de referencia | $\sum_{t \in A} w(t)$ | $\theta_2$ con $m = 1{,}2$ |
|---|---|---|
| 4 tags (el $A$ del ejemplo) | 6,40 | 0,375 |
| 10 tags | 15,0 | 0,16 |
| 2 tags | 2,0 | 1,0: tiene que coincidir en todo |

Un umbral fijo de 0,5 haría que un aviso con doce tags casi nunca encontrara similares y que uno con dos tags encontrara cualquier cosa. Si con $k = 2$ quedan menos de 5 avisos admitidos (`minSample`), se reintenta con $k = 1$ (`fallbackThresholdTags`) y la respuesta se marca `fallback`; si ni así llega a 5, la métrica devuelve `insufficientData` y el frontend dice qué falta en lugar de dibujar un número. Los avisos similares no se muestran en la aplicación: son la muestra sobre la que se calculan el *time-to-fill* y el salario de la posición.

**Candidato contra aviso** (salario esperado de pares, *benchmark*): *overlap* ponderado sin normalizar, porque el CV tiene muchas más tags que un aviso y Jaccard lo penalizaría,

$$O(A, C) = \sum_{t \in A \cap C} w(t), \qquad \theta_k = k \cdot m.$$

Valen la misma $k$ y el mismo *fallback*; como el *overlap* no está normalizado, el umbral es directamente $k \cdot m$. En ambos casos, los empates en score se desempatan por cantidad de skills compartidas.

## 6. Manejo de archivos

**Los bytes nunca pasan por un servicio de LYNQ.** `lynq-file-storage` guarda solo metadata en MySQL (`stored_files`: `id`, `file_name`, `content_type`, `s3_key`, `status`, `owner_user_id`) y delega los bytes a un bucket S3 privado (`lynq-oficial-file-storage`) mediante URLs pre-firmadas de **15 minutos**. Es el único servicio con credenciales de S3; `lynq-app-backend` guarda únicamente el id del archivo (`users`, `companies`, `user_resumes`, `user_application_job`).

El flujo de importar un CV, orquestado por el BFF, en cinco pasos:

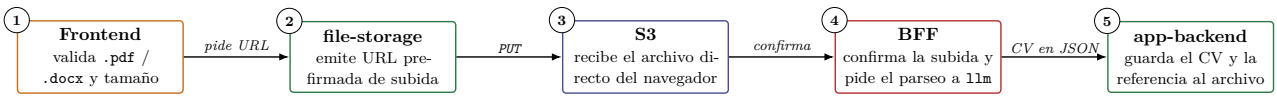



Figura 2: Importación de un CV. Los bytes van del navegador a S3 sin pasar por ningún servicio; el resto del flujo mueve solo ids, URLs pre-firmadas y el CV ya parseado.

- El frontend valida extensión (`.pdf`, `.docx`) y tamaño (10 MB) antes de pedir nada.
- File-storage registra el archivo como `PENDING` y devuelve una URL pre-firmada de subida.
- El navegador sube directo a S3 con esa URL.
- El BFF confirma la subida (file-storage verifica en S3 y pasa a `AVAILABLE`) y manda el archivo a `lynq-llm`, que lo convierte a JSON, detecta el idioma y extrae skills y tags.
- El backend guarda el CV, el idioma, las skills y la referencia al archivo. Si algo falla en el camino, el archivo se borra.

**Ownership.** El dueño sale del token resuelto contra el IAM. Solo él puede confirmar, consultar o borrar un archivo (403 en otro caso). Las descargas, en cambio, no se restringen al dueño a propósito: el id del archivo funciona como *capability* y es el backend quien decide qué ids mostrar (la empresa ve el CV con el que se postularon a su aviso). Los CVs adaptados por CV Tailor se renderizan a PDF con WeasyPrint en lynq-llm y se suben por el mismo mecanismo de URL `PUT` pre-firmada.

## 7. Eventos de dominio y analytics

**Analytics no lee la base del backend.** Escucha los eventos, los guarda crudos y los proyecta en un read model con la forma que las consultas necesitan. Eso corta el acople con el esquema transaccional y permite cambiar o agregar un chart reproyectando desde los eventos guardados.

### 7.1. Publicación

Hay siete tipos de evento sobre tres agregados: `JobPostPublished`, `JobPostUpdated`, `JobPostClosed`, `JobPostReopened`, `ApplicationSubmitted` (con el score ya calculado), `CandidateSkillsUpdated` y `CandidateExpectedSalaryUpdated`. Las decisiones de publicación:

- **Delivery *at-most-once*.** El servicio envía el evento a `DomainEventPublisher` dentro de la transacción y `DomainEventRelay` lo envía a SNS. Si el envío falla, el evento se pierde; de eso se ocupa el replay.
- **Ids determinísticos.** `eventId` es un UUIDv5 del tipo, el `aggregateId` y el hecho que el evento afirma (un cierre se identifica por sus fechas de apertura y cierre, una postulación por su id, las skills por el conjunto completo). Publicar dos veces el mismo hecho produce el mismo id.
- **Replay como reconciliación.** Lo que se pierde se recupera con `POST /internal/events/replay`, que recorre `job_posts`, `users` y `user_application_job` en páginas de 200 y reemite el estado actual en lotes de 10. Como los ids son determinísticos, lo que ya estaba no se guarda debido al `UNIQUE` del consumidor y lo nuevo se guarda; correrlo dos veces no afecta en nada.

### 7.2. Consumo y proyección

SNS (`lynq-domain-events`) reparte a la cola SQS `lynq-analytics-events`, con DLQ tras 5 intentos. El listener ejecuta, en **una sola transacción**: `INSERT` en `domain_events` (`event_id CHAR(36) UNIQUE`, `event_type`, `aggregate_type`, `aggregate_id`, `payload JSON`, `occurred_on`, `received_on`) y la proyección al read model (`job_posts`, `job_post_skills`, `job_post_tags`, `candidates`, `candidate_skills`, `candidate_tags`, `applications`). La idempotencia es el `UNIQUE` de `event_id`: un duplicado falla el insert y se hace *ack*. Los eventos fuera de orden se resuelven con marcas de tiempo por entidad (`details_occurred_on`, `status_occurred_on`, `skills_occurred_on`, `salary_occurred_on`): uno más viejo que lo ya proyectado se descarta pero queda en `domain_events`; un update que llega antes del publish se rechaza y SQS lo reintenta. Si la proyección falla, el evento hace rollback y vuelve a la cola.

### 7.3. Tres niveles de actualización

- **Nivel 1, al llegar el evento**: read model actualizado en segundos. El costo es consistencia eventual: el standing puede no reflejar la postulación recién hecha.
- **Nivel 2, al consultar**: las métricas por aviso se calculan en Java sobre el read model en cada request (decenas o cientos de filas) y se cachean en Redis una hora por endpoint y clave.
- **Nivel 3, snapshot diario**: un CronJob (08:00 UTC, después del feeder de las 06:00) llama a `POST /internal/snapshot`, que recalcula las frecuencias de tags, escribe una fila por día y dimensión en `job_daily_stats`, `skill_daily_demand` y `category_daily_salary` (upsert por clave compuesta: correrlo dos veces da lo mismo), y arma `candidate_daily_benchmark` pidiendo scores al backend en lote. Solo existe para los charts con fecha en el eje; si un día el job no corrió, el chart muestra datos faltantes en lugar de interpolar.

### 7.4. Métricas expuestas

| Endpoint (`/dmz/analytics`) | Rol | Qué calcula |
|---|---|---|
| `GET /job/{id}/time-to-fill` | COMPANY, dueño | Mediana, p25 y p75 (interpolación lineal) de los días entre apertura (o reapertura) y cierre de avisos similares. Excluye y cuenta aparte los `EXPIRED_BY_POLICY` (censurados a los 25 días sin verse). Con $N < 5$ devuelve la distribución global. |
| `GET /job/{id}/standing` | CANDIDATE, postulado | Sobre `applications.lynq_score`: rank $= 1 + \#\{\text{scores} > s\}$ y percentil $= 100 \cdot (\text{below} + \text{tied}/2)/\text{total}$; histograma por bandas; mediana del aviso si $N \geq 5$. Nunca expone la identidad de otros postulantes. |
| `GET /job/{id}/salary` | cualquiera | `positionSalary`: mediana, p25 y p75 del punto medio del rango de avisos similares, en la moneda del aviso. `peersExpectedSalary`: lo mismo sobre el salario esperado de candidatos similares. Mediana y no media: los salarios scrapeados mezclan períodos y traen errores. |
| `GET /candidate/me/benchmark` | CANDIDATE | Del snapshot: *market fit* (mediana del score contra los avisos relevantes), *reach* (% de avisos con score $\geq 60$), percentil y cuartiles entre pares, cobertura de skills, serie de 90 días y *skill unlocks*: cuántos avisos pasarían el umbral si tuviera cada skill faltante. |
| `GET /market?currency=` | cualquiera | Avisos abiertos y con salario, top 10 de skills con variación semanal, salario por categoría y modalidad, publicados por semana. |
| `GET /company/me/jobs` | COMPANY | Avisos propios con estado, postulaciones y score mediano. Es el único cache que se invalida por evento. |

Cuadro 2: Analíticas servidas por lynq-analytics.

## 8. CV Tailor: el agente de adaptación de CV

**El problema de diseño.** Un LLM que "mejora" un CV tiende a inventar: agrega una tecnología que pide el aviso, redondea ocho años a diez, traduce al idioma del chat. Un CV con una mentira es peor que un CV sin adaptar. La decisión fue **no pedirle al modelo que no invente, sino no darle con qué**: separar en agentes distintos la intención, la propuesta, el juicio y la escritura, de modo que ningún texto llegue al CV sin pasar por un juez con evidencia, y que la escritura la haga código, no un modelo.

### 8.1. El grafo

Cada turno de conversación se ejecuta como un grafo de LangGraph (Figura 3) sobre Amazon Bedrock (Nova Pro en producción; qwen2.5:7b en Ollama para desarrollo). El estado del turno (`TurnContext`) lleva el aviso, el CV base, el CV actual, los últimos cuatro pares del hilo, las recomendaciones del turno anterior y los *statements*: todo lo que el candidato dijo en la conversación más lo que ya confirmó.

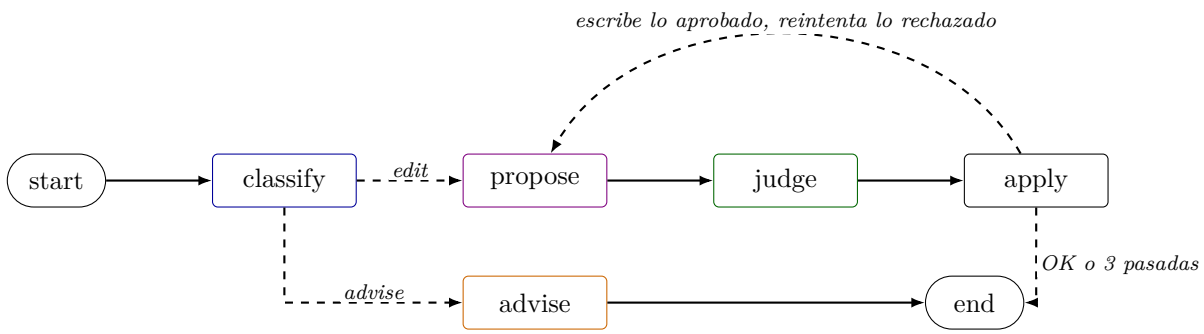



Figura 3: Grafo de un turno de CV Tailor. **classify**: agente de intención; **advise**: agente de recomendación; **propose**: agente de edición; **judge**: agente juez; **apply**: código, sin modelo. La rama `advise` no tiene ningún nodo que escriba: una consulta no puede modificar el CV por construcción.

1. **classify** (agente de intención). Una llamada, una palabra: `edit` o `advise`, leyendo el mensaje con los últimos cuatro del hilo. Un "sí" a una skill por la que CV Tailor preguntó es `edit`. Si el modelo falla o responde otra cosa, cae a `edit`: nunca se pierde un turno. Es la única decisión de ruteo que toma un modelo; puede correr en un modelo distinto (`BEDROCK_INTENT_MODEL_ID`).
2. **advise** (agente de recomendación). Devuelve `reply`, `warnings` y `recommendations` como datos `{id, section, entry, what}`; su schema no tiene campos para tocar el CV. Las recomendaciones se numeran y persisten en el mensaje para que "hacé la 2" del turno siguiente tenga referente.
3. **propose** (agente de edición). Recibe el aviso, el CV sin `personal_info`, los *statements* y la lista de *gaps*, y responde un `EditProposal` con *structured output*: `summary`, entradas de experiencia identificadas por empresa y puesto (nunca por índice) con `description` y `achievements`, los tres *buckets* de skills (`technical`, `tools`, `soft`), `confirmed` (lo que el candidato afirmó sobre sí mismo este turno) y `reply`. Sin tools, sin loop: un formulario.
4. **Guard de forma** (código, antes del juez). `plan` resuelve cada parte a su lugar en el CV base y rechaza sin consultar al modelo: `cut` si se pierden líneas, números o tecnologías; `moved` si una línea cambió de posición (Jaccard de tokens $\geq 0{,}75$ para considerarla la misma); `unbacked` si aparece un nombre propio o una skill que no está en el CV ni en lo que dijo el candidato; `unknown_entry` si la entrada no existe. Un *bucket* de skills puede crecer al final, nunca achicarse ni reordenarse.
5. **judge** (agente juez). Lee cada parte junto a su original, con líneas numeradas y contadas, más `candidate_statements` y `this_turn` (qué preguntó CV Tailor y qué contestó el candidato). Debe citar la evidencia antes de aprobar, copiada del CV o del candidato, nunca de la propuesta ni del aviso: pedir una skill no significa tenerla. Devuelve un `Verdict` por parte con `ok`, `kind` y `reason`; los `kind` de rechazo son `language`, `dropped_content`, `reordered`, `dropped_skill`, `invented`, `unsupported_skill`, `wording` y `padding`. Una parte que el juez omite queda `unjudged` y no se aplica.
6. **apply** (código). `commit` escribe en el CV solo lo aprobado y registra cada cambio. Si quedaron rechazos y quedan pasadas (`MAX_PASSES` = 3), el grafo vuelve a `propose` con el motivo de cada rechazo en el hilo; es la única vuelta atrás. La vuelta sale de `apply` y no del juez a propósito: lo aprobado se escribe en cada pasada, y la siguiente propuesta se hace sobre el CV ya actualizado y solo sobre lo rechazado. Al terminar, el candidato recibe el `reply` del modelo y, aparte, un aviso escrito por código con lo que no entró y por qué: el chat no promete lo que el CV no dice.

**Cómo sabe `apply` qué modificar.** Porque la propuesta no es texto libre sino un formulario: cada campo del `EditProposal` *es* una ubicación del CV. `summary` solo puede ir al resumen, una entrada de experiencia se identifica por empresa y puesto, y `skills.technical` es ese *bucket* y ningún otro. El modelo no puede pedir "cambiá la tercera línea" ni agregar una sección: el schema no tiene campos para eso. `plan` convierte el formulario en *partes*, cada una con su ruta en el JSON del CV, el texto original y el propuesto; el juez devuelve un veredicto por parte con el mismo id, y `commit` recorre la lista: escribe en su ruta las aprobadas y arma, con una plantilla, el aviso de las rechazadas. Un turno con dos partes:

| id | Ruta en el CV | Original | Propuesto | Veredicto |
|---|---|---|---|---|
| 1 | `work_experience[Acme].description` | Built services on Kubernetes. | Built services on Kubernetes. – Built the payment workers in Go | `ok` |
| 2 | `skills.technical` | Java, Postgres, Kubernetes | Java, Postgres, Kubernetes, Go | `ok` |

Si el modelo escribiera el CV final directamente, una parte rechazada no se podría separar de una aprobada, y nada garantizaría que no tocó `personal_info`, la educación o las fechas. Con partes direccionadas, lo rechazado queda afuera parte por parte, y lo que el schema no contempla es imposible de modificar.

**Gaps y evidencia del candidato.** Al crear la conversación, el código compara las skills del aviso (`extractedSkills`, vía `lynq-llm /skill-enhance`) con el CV y los *statements*, y arma el saludo sin llamar al modelo: lista hasta 4 gaps y pregunta en qué puesto y qué hizo. Una skill que el candidato confirma entra al *bucket* en el mismo turno; a una entrada de experiencia entra recién cuando dice dónde la usó y para qué ("una tecnología no se muda a un trabajo que nunca la usó"). Lo confirmado

se persiste en `conversation.confirmed` y vuelve como evidencia en todos los turnos siguientes, dentro o fuera de la ventana de contexto.

**Persistencia y traza.** `lynq_agent_db` guarda `conversation` (snapshots congelados del aviso y del CV, precios por token congelados, `run_token`, totales de costo), `message`, `resume_version` (una por turno que aplicó cambios, con su lista de `changes`) y `trace_span` (un span por llamada al modelo con tokens, costo y latencia, uno por pasada de `apply` con el veredicto). Un turno son dos transacciones cortas: *claim* (`FOR UPDATE`, pasa a `RUNNING` con un `run_token` nuevo) y *persist* (verifica el `run_token`; si no coincide, el resultado se descarta como `stale_run`); los agentes corren sin transacción abierta, con timeout de 600 s. El BFF expone el flujo en `/resume/{resumeId}/tailor/{jobId}`, `/tailor/{id}/turn` y `/tailor/{id}/apply`, que renderiza el CV adaptado como archivo y lo deja listo para postular.

## 9. Resumen de decisiones y deuda conocida

| Decisión | Por qué | Qué se paga |
|---|---|---|
| BFF como único Ingress, servicios en red privada | Un solo lugar donde verificar firma y relayar; superficie de ataque mínima | Una llamada más por request; el BFF conoce a todos |
| Validar el token contra el IAM en cada servicio | El secreto vive solo en IAM y BFF; los roles se releen | Latencia y dependencia del IAM en cada request |
| Bytes directos a S3 con URL pre-firmada | Ningún servicio recibe archivos; sin multipart ni límites de pod | El frontend valida tipo y tamaño; file-storage confía |
| Score sin pesos, máximo de dos coberturas | Explicable al candidato (“tenés 6 de 9”) y calculado en un solo lugar | No distingue skills centrales de accesorias |
| Similitud por tags ponderadas por rareza (IDF) | Comparar por concepto y no por palabra; umbral en “tags típicas” | Depende de la calidad de las tags del LLM |
| Sin outbox, replay como reconciliación | Backend liviano, sin job ni retención | Entrega at-most-once; se corre el replay antes de una demo |
| Read model propio en analytics | Cortar el acople con el esquema transaccional | Consistencia eventual de segundos |
| Métricas al vuelo, snapshot solo para series | Sin watermarks que se desvíen en silencio | Un hueco en la serie si el job no corrió |
| Agente con juez y escritura en código | Nada entra al CV sin evidencia citada | De 3 a 7 llamadas al modelo por turno de edición |

**Pendientes identificados.**[1]

| Pendiente | Estado |
|---|---|
| Login con Google (OAuth 2.0 / OIDC) | A DEFINIR |
| *Logout* con revocación del refresh token en Redis | A DEFINIR |
| Validación de tipo y tamaño de archivo del lado de file-storage | A DEFINIR |
| lynq-email: servicio de notificaciones por correo | A DEFINIR |
| IAM con OTP como segundo factor | A DEFINIR |
| Lynq CD: despliegue continuo | EN ROADMAP |
| Juez con un modelo distinto al del editor | EN ROADMAP |
| Agente de IA de búsqueda autónoma | FUERA DE ALCANCE |
| Outreach inteligente al talent pool | FUERA DE ALCANCE |
| Métricas de calidad de contratación | FUERA DE ALCANCE |

[1] Pendientes identificados al día de la fecha de este documento. Los ítems listados no son vinculantes ni implican obligación de implementarlos; su alcance y prioridad se definirán con los directores de carrera.